



ЛЕКЦІЯ №5

ОРГАНІЗАЦІЯ ЦИКЛІВ

Цикл for
Цикл while
Функція range()

ПЛАН

1. Поняття циклу
2. Види циклів
3. Цикл з перед умовою **while...**
4. Безумовний цикл **while...**
5. Переривання циклу – break
6. Цикл з лічильником **for...**
7. Функція **range**
8. Табулювання функції
9. Приклад: шифр Юлія Цезаря

ПОНЯТТЯ ЦИКЛУ

Вираз, котрий визначає чи буде в черговий раз виконуватися ітерація, або цикл завершиться, називається **умовою виходу** або умовою закінчення циклу.

Крім того більшість мов програмування надають **ресурси для дострокового завершення циклу (break)**, тобто виходу із циклу незалежно від істинності умови виходу.

Виконання будь-якого циклу включає первісну ініціалізацію змінних циклу, перевірку умови виходу, виконання тіла циклу й відновлення змінної циклу на кожній ітерації.

ВИДИ ЦИКЛІВ

У мові програмування Python є два основні види циклів: **for** та **while**.

Цикл for. Використовується для перебору (ітерації) елементів послідовностей, таких як рядки, списки, кортежі, множини, словники або числові діапазони, що створюються за допомогою функції `range()`. Блок коду виконується для кожного окремого елемента колекції.

Цикл while. Виконує блок коду доти, доки задана умова залишається істинною (**True**). Умова перевіряється перед кожною ітерацією. Якщо вона стає хибною, цикл припиняє свою роботу.

ЦИКЛ З ПЕРЕД УМОВОЮ

Цикл із передмовою – цикл, що виконується поки істинна деяка умова, зазначене перед його початком. Ця умова перевіряється до виконання тіла циклу, тому тіло може бути не виконано жодного разу (якщо умова із самого початку невірна). У більшості процедурних мов програмування реалізується оператором `while`, звідси його друга назва - `while`-цикл.

```
while умова:  
    тіло циклу
```

БЕЗУМОВНИЙ ЦИКЛ

Безумовні цикли.

Іноді в програмах використовуються цикли, вихід з яких не передбачений логікою програми. Такі цикли називаються безумовними, або нескінченними. Такі цикли створюються за допомогою конструкцій, призначених для створення звичайних (або умовних) циклів. Для забезпечення нескінченного повторення перевірка умови в такому циклі постійно виконуються, або заміняється константним значенням.

```
while True:  
    print('Hello, world!')
```

ПЕРЕРИВАННЯ ЦИКЛУ - BREAK

За допомогою оператора **break** ми можемо зупинити цикл, навіть якщо умова **while** виконується.

```
x = 1
while x < 6:
    print(x)
    if x == 3:
        break
    x+= 1
```

ЦИКЛ З ЛІЧИЛЬНИКОМ

Цикл із лічильником – цикл, у якому деяка змінна змінює своє значення від заданого початкового значення до кінцевого значення з деяким кроком, і для кожного значення цієї змінної тіло циклу виконується один раз.

У більшості процедурних мов програмування реалізується оператор **for**, у якому вказується лічильник (так звана «змінна циклу»), необхідне кількість проходів (або граничне значення лічильника) і, можливо, крок, з яким змінюється лічильник.

ІНСТРУКЦІЯ ЦИКЛУ FOR

У циклі **for** вказується змінна і множина значень, яким буде пробігати змінна. Множина значень може бути задано: списком, кортежем, рядком або діапазоном.

for змінна in множина значень:

```
1 for x in "banana":  
2     print(x)
```

```
PS D:\Python> python u1.py  
b  
a  
n  
a  
n  
a
```

ПЕРЕБІР СИМВОЛІВ РЯДКА

Оскільки рядки є масивами, ми можемо прокручувати символи в рядку за допомогою **for** циклу.

У циклі **for** вказується змінна і множина значень, яким буде пробігати змінна. Множина значень може бути задано списком, кортежем, рядком або діапазоном.

for змінна in множина:

```
1 for x in "banana":  
2     print(x)
```

```
PS D:\Python> python u1.py  
b  
a  
n  
a  
n  
a
```

ВИКОРИСТАННЯ СПИСКУ

СПИСОК

Списки використовуються для зберігання кількох елементів в одній змінній.

Списки є одним із 4 вбудованих типів даних у Python, які використовуються для зберігання колекцій даних, інші 3 є Tuple , Set і Dictionary , усі з різною якістю та використанням.

Списки створюються за допомогою квадратних дужок:

```
1 fruits = ["apple", "banana", "cherry"]
2 for x in fruits:
3     print(x)
```

```
PS D:\Python> python u1.py
apple
banana
cherry
```

ФУНКЦІЯ RANGE

Досить часто при розробці програм необхідно отримати послідовність (діапазон) цілих чисел:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

В Python передбачена функція `range`, що створює послідовність (діапазон) чисел. В якості аргументів функція приймає: початкове значення діапазону (за замовчуванням 0), кінцеве значення (**НЕ включно**) і крок (за замовчуванням 1)

```
1 for i in range(0, 10, 1):  
2     print(i, end=' ')
```

```
PS D:\Python> python u1.py  
0 1 2 3 4 5 6 7 8 9
```

ПРИКЛАД #1

Скласти програму обчислення суми дробів

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9} + \frac{1}{10}$$

1. Чисельник рівний 1
2. Знаменник змінюється від 2 до 10 на 1
3. Позначимо знаменник через x
4. Загальна формула дробу $1/x$

$$\sum_{x=2}^{10} \frac{1}{x}$$

ПРИКЛАД #1 $\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9} + \frac{1}{10}$

```
1 S=0
2 for i in range(2, 11):
3     S+=1/i
4 print(S)
```

```
PS D:\Python> python u1.py
1.928968253968254
```

ТАБУЛЮВАННЯ ФУНКЦІЇ

Табулювання функції – обчислення значення деякої функції $f(x)$ при рівномірному розподілі значень змінної x на відрізок від a до b .

Табуляційна таблиця складає дві колонки значень.

Перша колонка значення змінної x , друга – обчислені значення функції $f(x)$ у відповідних точках значення x .

x	$f(x)$
x_1	$f(x_1)$
x_2	$f(x_2)$
x_3	$f(x_3)$

ПРИКЛАД #2

Виконайте перетворення значення температури у градусах Цельсія (C) (від 15 до 30) для температурної шкали Фаренгейта (F). Програма повинна відображати еквівалентну температуру у градусах Фаренгейта ($F = 32 + 9/5 * C$).

```
1 print("-"*23)
2 print("Цельсій    | Фаренгейт")
3 for t in range(15,31):
4     f=32*9/5*t
5     print(f"{t:10d} | {f:10.2f}")
6 print("-"*23)
```

Цельсій	Фаренгейт
15	864.00
16	921.60
17	979.20
18	1036.80
19	1094.40
20	1152.00
21	1209.60
22	1267.20
23	1324.80
24	1382.40
25	1440.00
26	1497.60
27	1555.20
28	1612.80
29	1670.40
30	1728.00

ПРИКЛАД #3

Організувати неперервне введення чисел з клавіатури, доки користувач не введе 0. Після введення нуля, показати на екран кількість чисел, які були введені, їх загальну суму та середнє арифметичне.

```
1  S=0
2  n=0
3  x=1
4  while x!=0:
5      x=int(input())
6      S+=x
7      n+=1
8  print(f"Кількість введених чисел {n-1:d}")
9  print(f"Сума введених чисел {S:d}")
10 print(f"Середньоарифметичне чисел {S/(n-1):.2f}")
```

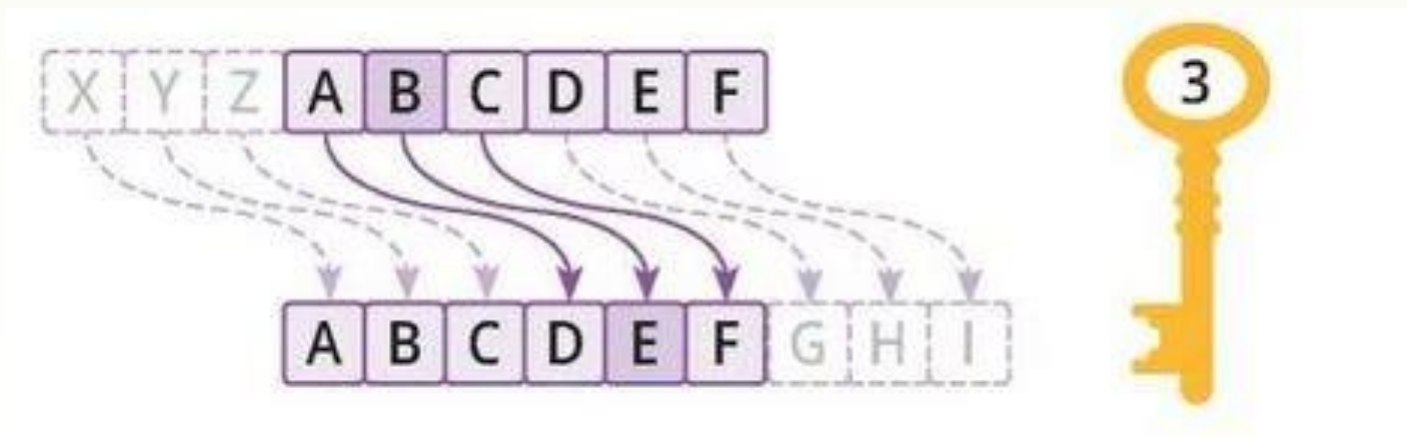
```
2
6
9
0
Кількість введених чисел 3
Сума введених чисел 17
Середньоарифметичне чисел 5.67
```

ШИФР ЦЕЗАРЯ

Юлій Цезар використовував адитивний шифр, щоб зв'язатися зі своїми чиновниками. Із цієї причини адитивні шифри згадуються іноді як шифри Цезаря. Цезар для свого зв'язку використовував цифру 3.

Принцип дії полягає в тому, щоб циклічно зсунути алфавіт, а ключ — це кількість літер, на які робиться зсув.

apple
dssoh



ШИФР ЦЕЗАРЯ

1. Визначити код символу
2. Додати до коду 3
3. Визначити символ за кодом

Виняток:

120, 121, 122

Вирішення:

$123 - 26 = 97$

$124 - 26 = 98$

$124 - 26 = 99$

Бінарне	Дес.	Шіст.	Графічне
0110 0000	96	60	`
0110 0001	97	61	a
0110 0010	98	62	b
0110 0011	99	63	c
0110 0100	100	64	d
0110 0101	101	65	e
0110 0110	102	66	f
0110 0111	103	67	g
0110 1000	104	68	h
0110 1001	105	69	i
0110 1010	106	6A	j
0110 1011	107	6B	k
0110 1100	108	6C	l
0110 1101	109	6D	m
0110 1110	110	6E	n
0110 1111	111	6F	o
0111 0000	112	70	p
0111 0001	113	71	q
0111 0010	114	72	r
0111 0011	115	73	s
0111 0100	116	74	t
0111 0101	117	75	u
0111 0110	118	76	v
0111 0111	119	77	w
0111 1000	120	78	x
0111 1001	121	79	y
0111 1010	122	7A	z

Бінарне	Дес.	Шіст.	Графічне
0110 0000	96	60	`
0110 0001	97	61	a
0110 0010	98	62	b
0110 0011	99	63	c
0110 0100	100	64	d
0110 0101	101	65	e
0110 0110	102	66	f
0110 0111	103	67	g
0110 1000	104	68	h
0110 1001	105	69	i
0110 1010	106	6A	j
0110 1011	107	6B	k
0110 1100	108	6C	l
0110 1101	109	6D	m
0110 1110	110	6E	n
0110 1111	111	6F	o
0111 0000	112	70	p
0111 0001	113	71	q
0111 0010	114	72	r
0111 0011	115	73	s
0111 0100	116	74	t
0111 0101	117	75	u
0111 0110	118	76	v
0111 0111	119	77	w
0111 1000	120	78	x
0111 1001	121	79	y
0111 1010	122	7A	z

ШИФР ЦЕЗАРЯ

```
word=input()  
word_code=""  
#шифрування  
for i in word:  
    x=ord(i)+3  
    if(x>122):  
        x-=26  
    word_code+=chr(x)  
print(word_code)
```

```
PS D:\Python> & C:  
apple  
dssoh
```

```
word=input("Word ")  
n=int(input("code "))  
word_code=""  
for i in word:  
    x=ord(i)+n  
    if(x>122):  
        x-=26  
    word_code+=chr(x)  
print(word_code)
```

```
PS D:\Python> & C:  
Word apple  
code 20  
ujjfy
```